

|                        |                                                |
|------------------------|------------------------------------------------|
| ДИСЦИПЛИНА             | Программирование в операционной системе Линукс |
| ИНСТИТУТ               | Передовая инженерная школа СВЧ-электроники     |
| КАФЕДРА                | Передовых технологий                           |
| ВИД УЧЕБНОГО МАТЕРИАЛА | Методические указания по дисциплине            |
| ПРЕПОДАВАТЕЛЬ          | Астафьев Рустам Уралович                       |
| СЕМЕСТР                | 1 семестр, 2025/2026 уч. год                   |

Ссылка на материал:

<https://github.com/astafiev-rustam/programming-in-the-linux-operating-system/tree/lecture-1-2>

## Лекция №2: Управление процессами, правами и пользователями

### Теоретическая вводная

#### Процессы в Linux: основа всего

Когда вы работаете в Linux, каждая запущенная программа — будь то простой текстовый редактор или сложный веб-сервер — представляет собой процесс. Процесс можно представить как живую, дышащую сущность внутри операционной системы. Это не просто файл на диске, а активный экземпляр программы, который получает от системы собственные ресурсы: память, процессорное время, доступ к файлам и устройствам.

Каждому процессу система присваивает уникальный паспорт — PID (Process ID). Это числовой идентификатор, по которому система отличает один процесс от другого. Но процессы редко рождаются сами по себе. У каждого процесса есть родитель — тот процесс, который его создал. Этот родительский процесс имеет свой собственный идентификатор PPID (Parent Process ID). Если проследить всю цепочку родителей до самого начала, мы всегда придем к первому процессу системы — `init` или в современных дистрибутивах `systemd` с PID 1. Это корень всего процессного дерева, прародитель всех остальных процессов.

Особую категорию процессов составляют демоны — фоновые процессы, которые работают независимо от пользовательского интерфейса. Они похожи на невидимых слуг системы: демон SSH обеспечивает удаленное подключение, демон веб-сервера обрабатывает запросы к сайту, демон базы данных управляет хранением информации. Они тихо работают в фоне, выполняя свою работу без нашего непосредственного вмешательства.

#### Сигналы: язык общения с процессами

Представьте, что процесс — это человек, занятый своей работой. Как мы можем с ним взаимодействовать? Для этого в Linux существуют сигналы — специальные сообщения, которые мы можем отправить процессу. Это своего рода язык жестов в мире операционных систем.

Самые важные сигналы, которые нужно знать: SIGTERM — это вежливая просьба завершить работу. Процесс получает такое сообщение и может корректно завершить свои дела: сохранить данные, закрыть файлы, попрощаться с другими процессами. SIGKILL — это немедленное и безоговорочное "убийство". Процесс не может его проигнорировать или отложить — он прекращает существование мгновенно. SIGHUP часто используется для того, чтобы сказать демону: "Перечитай свою конфигурацию, я в нее что-то поменял".

## Пользователи и группы: система безопасности Linux

Linux с самого начала создавалась как многопользовательская система. Это значит, что одновременно с компьютером могут работать несколько человек, и система должна обеспечивать, чтобы они не мешали друг другу. Каждый пользователь имеет свой уникальный идентификатор (UID) и может входить в одну или несколько групп.

Особое место занимает пользователь root — суперпользователь, обладающий неограниченной властью в системе. Это как мастер-ключ, который открывает все двери. Обычные пользователи имеют ограниченные права и могут работать только с теми файлами и программами, к которым им явно разрешили доступ.

## Права доступа: кто что может делать

Система прав доступа в Linux элегантна и проста. Каждый файл и каталог имеет три уровня доступа: права для владельца (пользователя, который создал файл), для группы (набора пользователей) и для всех остальных.

Для каждого из этих уровней определяются три типа разрешений: чтение (r) — возможность посмотреть содержимое файла или списка файлов в каталоге; запись (w) — возможность изменить файл или создать новый в каталоге; выполнение (x) — возможность запустить файл как программу или войти в каталог.

Эти права отображаются в виде знакомой строки из 10 символов, которую вы видите при выполнении команды `ls -l`. Первый символ показывает тип (например, `-` для обычного файла, `d` для каталога), а остальные девять — три группы по три символа, показывающие права для владельца, группы и остальных.

Например, права `-rwxr-xr--` говорят нам: это обычный файл, владелец может читать, писать и выполнять его; члены группы могут читать и выполнять; все остальные могут только читать. Это не просто абстрактные символы — это реальные правила, которые система применяет каждый раз, когда кто-то пытается получить доступ к файлу.

Понимание этой системы — ключ к эффективной работе в Linux. Это основа безопасности, стабильности и предсказуемости поведения системы.

---

## Практические примеры

### Пример 1: Наблюдение за процессами в реальном времени

**Цель:** Понять, как использовать `top` и `htop` для мониторинга системы.

```
# 1. Запустим утилиту top. Она покажет динамически обновляемую информацию о процессах и ресурсах системы.
```

```
top
```

```
# Внутри top:
```

```
# - Нажмите `q` для выхода.
```

```
# - Нажмите `M` (Shift+m) для сортировки процессов по использованию памяти.
```

```
# - Нажмите `P` (Shift+p) для сортировки по загрузке CPU.
```

```
# - Обратите внимание на столбцы: PID, USER, %CPU, %MEM, COMMAND.
```

```
# 2. Выйдем из top и установим более продвинутую альтернативу - htop.
```

```
# Команда sudo выполняет следующую команду с правами root.
```

```
sudo apt update && sudo apt install htop
```

```
# 3. Запустим htop. Он имеет более удобный интерфейс и цветовую схему.
```

```
htop
```

```
# Внутри htop:
```

```
# - Используйте стрелки `вверх/вниз` для навигации.
```

```
# - Нажмите `F9` для отправки сигнала процессу (откроется меню выбора сигнала).
```

```
# - Нажмите `F10` для выхода.
```

## Пример 2: Управление фоновыми и интерактивными задачами

**Цель:** Научиться управлять выполнением процессов в оболочке Bash.

```
# 1. Запустим процесс в foreground (интерактивно). Он заблокирует терминал.  
sleep 60
```

```
# 2. Прервём его выполнение, нажав комбинацию клавиш Ctrl+C.
```

```
# 3. Теперь запустим процесс в background (фоне), добавив амперсанд & в конец команды.
```

```
sleep 120 &
```

```
# Обратите внимание на вывод: [1] 1234, где 1 - номер задачи (job), 1234 - PID процесса.
```

```
# Терминал не заблокирован, мы можем продолжать работу.
```

```
# 4. Посмотрим список текущих задач (jobs) в нашей оболочке.
```

```
jobs
```

```
# 5. Вернём фоновую задачу в foreground.
```

```
fg %1
```

```
# Процесс снова стал интерактивным и заблокировал терминал. Нажмём Ctrl+Z, чтобы приостановить его.
```

```
# 6. Команда Ctrl+Z посылает сигнал SIGTSTP, который приостанавливает процесс.
```

```
# Он переходит в состояние "Stopped". Посмотрим на это в jobs.
```

```
jobs
```

```
# 7. Снова отправим приостановленную задачу в фон. Она продолжит выполнение.
bg %1

# 8. Завершим фоновый процесс, отправив ему сигнал SIGTERM.
# Сначала узнаем его PID (если не помним). Команда grep ищет строки, содержащие "sleep".
ps aux | grep sleep
kill 1234 # Замените 1234 на реальный PID из вывода предыдущей команды.

# 9. Проверим, что процесс завершился. Команда jobs должна показать "Завершён".
jobs
```

### Пример 3: Поиск и принудительное завершение процессов

**Цель:** Научиться находить конкретные процессы и управлять их состоянием с помощью сигналов.

```
# 1. Запустим несколько фоновых процессов.
sleep 300 &
sleep 400 &
tail -f /var/log/syslog & # tail будет постоянно читать системный лог.

# 2. Найдём все процессы sleep.
ps aux | grep sleep

# 3. Завершим ВСЕ процессы sleep "вежливым" способом (SIGTERM).
# Команда pkill ищет процессы по имени и отправляет им сигнал.
pkill sleep

# 4. Проверим, что они завершились.
jobs
ps aux | grep sleep

# 5. Теперь попробуем завершить процесс tail. Он может быть устойчив к SIGTERM.
# Сначала попробуем SIGTERM (по умолчанию).
pkill tail

# 6. Если он не завершился (проверьте через jobs), применим SIGKILL.
# Сигнал SIGKILL нельзя перехватить или проигнорировать.
pkill -9 tail

# 7. Альтернативный способ завершить процесс по PID (если он известен).
# kill -9 <PID>
```

### Пример 4: Управление правами доступа к файлам (chmod)

**Цель:** Научиться изменять права доступа с помощью символьной и числовой (восьмеричной) записи.

```
# 1. Создадим тестовый файл и посмотрим его текущие права.
touch myfile.txt
ls -l myfile.txt
# Права, скорее всего, будут -rw-r--r-- (644).

# 2. Сделаем файл исполняемым для владельца (пользователя).
chmod u+x myfile.txt
ls -l myfile.txt # Теперь права: -rwxr--r-- (744).

# 3. Добавим право на запись для группы.
chmod g+w myfile.txt
ls -l myfile.txt # Теперь права: -rwxrw-r-- (764).

# 4. Запретим чтение файла для всех остальных пользователей.
chmod o-r myfile.txt
ls -l myfile.txt # Теперь права: -rwxrw---- (760).

# 5. Теперь используем ВОСЬМЕРИЧНУЮ (числовую) нотацию для настройки прав.
# Каждая цифра - это сумма: r=4, w=2, x=1.
# Владелец: rwx = 4+2+1 = 7
# Группа: rw- = 4+2+0 = 6
# Остальные: r-- = 4+0+0 = 4
chmod 764 myfile.txt
ls -l myfile.txt # Права должны остаться -rwxrw-r--.

# 6. Установим часто используемые права для скрипта: только владелец может читать,
писать и выполнять.
chmod 700 myfile.txt
ls -l myfile.txt # Права: -rwx-----

# 7. Установим часто используемые права для конфигурационного файла: владелец
может читать и писать, остальные - только читать.
chmod 644 myfile.txt
ls -l myfile.txt # Права: -rw-r--r--
```

## Пример 5: Управление владельцем файла и группой (chown, chgrp)

**Цель:** Научиться изменять владельца и группу файлов.

```
# 1. Посмотрим на текущего владельца и группу нашего файла.
ls -l myfile.txt

# 2. Создадим новую группу (требуется права root).
sudo groupadd mytestgroup

# 3. Сменим группу-владельца файла на только что созданную.
sudo chgrp mytestgroup myfile.txt
ls -l myfile.txt # Убедимся, что группа изменилась.

# 4. Команда chown позволяет менять и владельца, и группу одновременно.
# Синтаксис: chown владелец:группа файл
```

```
# Создадим нового пользователя (требуется права root).
sudo adduser testuser

# 5. Сменим владельца файла на testuser, а группу оставим mytestgroup.
sudo chown testuser:mytestgroup myfile.txt
ls -l myfile.txt # Убедимся, что владелец и группа изменились.

# 6. Сменим только владельца.
sudo chown vagrant myfile.txt # Вернем файл себе (замените vagrant на ваше имя пользователя).
ls -l myfile.txt

# 7. Рекурсивно изменим владельца и группу для целого каталога.
mkdir mydir
touch mydir/file1.txt mydir/file2.txt
sudo chown -R testuser:mytestgroup mydir/
ls -l mydir/ # Убедимся, что права поменялись у всех файлов внутри.

# 8. (Очистка) Удалим созданных пользователя и группу.
sudo deluser testuser
sudo delgroup mytestgroup
```

## Пример 6: Понимание UID, GID и эффективных прав

**Цель:** Научиться смотреть идентификаторы пользователей и групп, понимать, как назначаются права.

```
# 1. Посмотрим на нашего текущего пользователя и его группы
id
# Вывод покажет: uid=1000(user), gid=1000(user), groups=1000(user),4(adm),...
# Это значит:
# uid - ваш идентификатор пользователя
# gid - идентификатор вашей основной группы
# groups - список всех групп, в которых вы состоите

# 2. Создадим файл и посмотрим его права
touch test_file.txt
ls -l test_file.txt
# Владелцем будет ваш текущий пользователь, группой - ваша основная группа

# 3. Создадим нового пользователя и новую группу
sudo adduser testuser2
sudo groupadd testgroup2

# 4. Добавим нашего текущего пользователя в новую группу
sudo usermod -a -G testgroup2 $USER
# Команда $USER подставит имя текущего пользователя

# 5. Выйдем и зайдём заново, чтобы изменения в группах вступили в силу
# (Внимание: это закроет текущую сессию SSH!)
sudo su - $USER
```

## Пример 7: Глубокий анализ процессов с помощью ps

**Цель:** Научиться использовать расширенные возможности ps для получения детальной информации.

```
# 1. Базовая информация о процессах текущего пользователя
ps

# 2. Полная информация о всех процессах в системе
ps aux
# Столбцы: USER, PID, %CPU, %MEM, VSZ, RSS, TTY, STAT, START, TIME, COMMAND

# 3. Детальная информация о конкретном процессе (например, sshd)
ps -p $(pgrep sshd) -f
# Или альтернативный вариант:
ps aux | grep sshd

# 4. Посмотрим дерево процессов - какие процессы кого породили
ps auxf
# Или используем pstree (может потребовать установки)
sudo apt install pstree
pstree

# 5. Посмотрим процессы, использующие много памяти (сортировка по %MEM)
ps aux --sort=-%mem | head -10

# 6. Посмотрим процессы, использующие много CPU (сортировка по %CPU)
ps aux --sort=-%cpu | head -10
```

## Пример 8: Работа с приоритетами процессов (nice и renice)

**Цель:** Научиться управлять приоритетом выполнения процессов.

```
# 1. Запустим процесс с низким приоритетом (высокое значение nice)
nice -n 10 sleep 100 &
# Nice value может быть от -20 (высший приоритет) до 19 (низший приоритет)

# 2. Посмотрим приоритет процесса
ps -l -p $(pgrep sleep)
# В столбце NI увидим значение nice (10)

# 3. Запустим процесс с высоким приоритетом (требуется права root)
sudo nice -n -10 sleep 200 &

# 4. Проверим приоритет
ps -l -p $(pgrep sleep | tail -1)

# 5. Изменим приоритет уже running процесса
# Сначала запустим обычный процесс
sleep 300 &
# Узнаем его PID
```

```
sleep_pid=$(pgrep sleep | tail -1)

# 6. Понизим его приоритет
renice -n 15 $sleep_pid

# 7. Проверим изменение
ps -l -p $sleep_pid

# 8. Убедимся, что все sleep процессы завершены
pkill sleep
```

## Пример 9: Поиск файлов по правам доступа

**Цель:** Научиться находить файлы с определенными правами доступа.

```
# 1. Создадим тестовые файлы с разными правами
touch /tmp/file1.txt /tmp/file2.txt /tmp/file3.txt
chmod 755 /tmp/file1.txt      # rwxr-xr-x
chmod 700 /tmp/file2.txt      # rwx-----
chmod 644 /tmp/file3.txt      # rw-r--r--

# 2. Найдем все файлы с правами 755 в /tmp
find /tmp -type f -perm 755 2>/dev/null

# 3. Найдем файлы, которые исполняются для всех (others have execute)
find /tmp -type f -perm -o=x 2>/dev/null

# 4. Найдем файлы, где группа имеет право на запись
find /tmp -type f -perm -g=w 2>/dev/null

# 5. Найдем файлы с SETUID битом (опасные!)
find /usr/bin -type f -perm -4000 2>/dev/null | head -5

# 6. Найдем файлы с SETGID битом
find /usr/bin -type f -perm -2000 2>/dev/null | head -5

# 7. Найдем файлы с открытым для записи sticky bit (чаще всего каталоги like /tmp)
find /tmp -type d -perm -1000 2>/dev/null
```

## Пример 10: Sticky bit и специальные биты прав доступа

**Цель:** Понять назначение и работу специальных битов SETUID, SETGID и Sticky bit.

```
# 1. Создадим общий каталог и установим sticky bit (аналогично /tmp)
sudo mkdir /shared_dir
sudo chmod 1777 /shared_dir
ls -ld /shared_dir
# Обратите внимание на 't' в конце прав: drwxrwxrwt
```



```
# 2. Sticky bit означает, что удалить файл может только его владелец
# Создадим два файла от разных пользователей
sudo su - testuser2 -c "touch /shared_dir/file_from_testuser2"
touch /shared_dir/file_from_me

# 3. Попробуем удалить файл другого пользователя (не должно получиться)
sudo su - testuser2 -c "rm /shared_dir/file_from_me"
# Получим ошибку: Operation not permitted

# 4. SETUID пример - посмотрим на /usr/bin/passwd
ls -l /usr/bin/passwd
# Права: -rwsr-xr-x. Буква 's' вместо 'x' у владельца - это SETUID

# 5. SETGID пример - создадим каталог с SETGID
mkdir team_dir
sudo chgrp testgroup2 team_dir
chmod 2775 team_dir
ls -ld team_dir
# Буква 's' в правах группы: drwxrwsr-x

# 6. SETGID заставляет новые файлы наследовать группу каталога
touch team_dir/test_file.txt
ls -l team_dir/test_file.txt
# Группа файла будет testgroup2, а не ваша основная группа
```

## Пример 11: Мониторинг системных ресурсов процессами

**Цель:** Научиться использовать утилиты для мониторинга использования ресурсов.

```
# 1. Установим дополнительные утилиты мониторинга
sudo apt install htop iotop iftop

# 2. Запустим htop для общего мониторинга
htop
# Нажмите F1 для помощи, F2 для настройки, F9 для отправки сигналов

# 3. В другом терминале посмотрим использование диска процессами
sudo iotop
# Покажет процессы, активно работающие с диском

# 4. Посмотрим сетевую активность
sudo iftop
# Покажет сетевой трафик между хостами

# 5. Мониторинг использования памяти в реальном времени
watch free -h
# Будет обновлять информацию каждые 2 секунды

# 6. Нагрузка на диск
iostat -x 1
# Покажет статистику ввода-вывода
```

```
# 7. Выйдем из всех утилит мониторинга (Ctrl+C в каждом терминале)
```

## Пример 12: Работа с системными сервисами через systemctl

**Цель:** Научиться управлять системными сервисами (демонами).

```
# 1. Посмотрим статус всех сервисов
sudo systemctl status

# 2. Посмотрим статус конкретного сервиса (например, SSH)
sudo systemctl status ssh

# 3. Остановим сервис (осторожно! можем потерять подключение)
# ВМЕСТО SSH используем сервис, который не критичен, например, cron
sudo systemctl stop cron
sudo systemctl status cron

# 4. Запустим сервис обратно
sudo systemctl start cron

# 5. Перезапустим сервис (stop + start)
sudo systemctl restart cron

# 6. Перезагрузим конфигурацию без остановки сервиса
sudo systemctl reload cron

# 7. Включим автозагрузку сервиса при старте системы
sudo systemctl enable cron

# 8. Запретим автозагрузку
sudo systemctl disable cron

# 9. Посмотрим, какие сервисы запускаются автоматически
sudo systemctl list-unit-files --type=service | grep enabled
```

## Пример 13: Анализ использования ресурсов конкретным процессом

**Цель:** Научиться анализировать, какие ресурсы использует конкретный процесс.

```
# 1. Запустим "тяжелый" процесс для тестирования
dd if=/dev/zero of=/tmp/testfile bs=1M count=100 &

# 2. Найдем PID процесса dd
dd_pid=$(pgrep dd)

# 3. Посмотрим статистику по процессу
ps -p $dd_pid -o pid,ppid,user,%cpu,%mem,rss,vsz,command
```

```
# 4. Посмотрим, какие файлы открыты процессом
sudo lsof -p $dd_pid

# 5. Мониторинг использования памяти процессом в реальном времени
watch "ps -p $dd_pid -o pid,%cpu,%mem,rss,vsz,command"

# 6. Используем /proc filesystem для детальной информации
cat /proc/$dd_pid/status | head -20

# 7. Посмотрим использование ввода-вывода
sudo iotop -p $dd_pid

# 8. Дождемся завершения процесса dd и удалим тестовый файл
wait
rm /tmp/testfile
```

## Пример 14: Работа с контрольными группами (cgroups)

**Цель:** Познакомиться с основами работы cgroups для ограничения ресурсов.

```
# 1. Посмотрим, какие cgroups доступны в системе
ls /sys/fs/cgroup/

# 2. Создадим свою cgroup для ограничения памяти
sudo mkdir /sys/fs/cgroup/memory/test_group

# 3. Установим лимит памяти в 100MB для нашей cgroup
echo "100M" | sudo tee /sys/fs/cgroup/memory/test_group/memory.limit_in_bytes

# 4. Добавим текущий shell в нашу cgroup
echo $$ | sudo tee /sys/fs/cgroup/memory/test_group/cgroup.procs

# 5. Запустим процесс, который попытается использовать много памяти
# Этот процесс будет ограничен нашей cgroup
python3 -c "
import array
try:
    # Попробуем аллоцировать 150MB (больше нашего лимита)
    data = array.array('b', [0]) * 150 * 1024 * 1024
    print('Память выделена')
except MemoryError:
    print('MemoryError: Не удалось выделить память!')
"

# 6. Убедимся, что процесс был убит из-за нехватки памяти
echo $?

# 7. Удалим нашу тестовую cgroup
sudo rmdir /sys/fs/cgroup/memory/test_group
```

## Пример 15: Продвинутое управление сигналами и отладка

**Цель:** Научиться использовать сигналы для отладки и управления процессами.

```
# 1. Создадим простой скрипт, который обрабатывает сигналы
cat > signal_handler.sh << 'EOF'
#!/bin/bash
echo "PID скрипта: $$"

# Функция для обработки SIGTERM
cleanup() {
    echo "Получен SIGTERM, выполняю очистку..."
    sleep 2
    echo "Очистка завершена, выхожу."
    exit 0
}

# Функция для обработки SIGUSR1
user_signal() {
    echo "Получен пользовательский сигнал SIGUSR1"
}

# Назначаем обработчики сигналов
trap cleanup SIGTERM
trap user_signal SIGUSR1

echo "Скрипт запущен. Отправьте сигналы:"
echo "  kill -TERM $$   для graceful shutdown"
echo "  kill -USR1 $$   для пользовательского сигнала"
echo "  kill -9 $$      для принудительного завершения"

# Бесконечный цикл
while true; do
    echo "Работаю... $(date)"
    sleep 10
done
EOF

# 2. Дадим права на выполнение и запустим скрипт в фоне
chmod +x signal_handler.sh
./signal_handler.sh &
script_pid=$!

# 3. Дадим скрипту немного поработать
sleep 3

# 4. Отправим пользовательский сигнал SIGUSR1
kill -USR1 $script_pid

# 5. Подождем и отправим "вежливый" SIGTERM
sleep 3
kill -TERM $script_pid
```

```
# 6. Посмотрим, как скрипт обработал сигналы
wait $script_pid
echo "Скрипт завершился с кодом: $?"

# 7. Теперь протестируем SIGKILL
./signal_handler.sh &
script_pid=$!
sleep 2
kill -9 $script_pid
wait $script_pid
echo "После SIGKILL код возврата: $?"

# 8. Очистка
rm signal_handler.sh
```